Mini Project Report

On

AI-Powered Online Examination and Assessment System

(Python, Flask, MySQL and Gemini API)

SUBMITTED BY:
Puneet Gupta
UID: 26MCI10246
CLASS: MCA (AI & ML)
SECTION: 24MAM4-4

SUBMITTED TO:
Python Programming Faculty
Advanced Internet Programming (AIP) Faculty

Table of Content

S. No.	Name	Page no.
1	Introduction	4
2	Background	4
3	Objectives	4
4	Technologies Used	4
5	Requirements	5
6	System Design	5
7	Source Code & Output	6
8	Functionalities	12
9	Conclusion	12
10	Future Work	12
11	Learning Outcomes	12
12	GitHub Link	13
13	Bibliography	13

Acknowledgement

I would like to express my gratitude to the faculty of Python Programming and Advanced Internet Programming for their guidance and feedback during the development of this project.

I am thankful to the Department of Computer Applications, Chandigarh University, for providing the learning environment and resources that supported this work. I also thank my classmates who tested the student and faculty workflows and helped identify issues in the application.

Finally, I would like to thank my family for their patience and support throughout the project.

Puneet Gupta
(26MCI10246)

1. Introduction

This report describes ExamAI, an online examination and assessment system implemented in Python. It provides separate student and teacher workflows: faculty create and publish papers, students complete timed assessments, and the application calculates each result from the saved answer key.

The project also uses generative AI for two language tasks. Gemini can draft multiple-choice questions for a teacher to review, and it can prepare a learning report from a student's actual incorrect answers. AI is not allowed to award marks. The Python application performs deterministic scoring and stores the result in MySQL.

The same application is suitable for both Python Programming and Advanced Internet Programming (AIP): Python and Flask are the core implementation, while the Gemini API demonstrates an applied AI feature in a real web workflow.

2. Background

Preparing a question paper and checking each response manually take time. A score by itself also gives a student little guidance about which concepts to revisit. Generative AI can speed up question drafting and produce explanations, but its answer keys may be wrong or its feedback may not match the student's actual responses.

ExamAI addresses these risks with three separate responsibilities. Gemini proposes content; the teacher checks generated questions before approval; and the application compares submitted choices with the stored answer key. Feedback is built from the attempt's real wrong answers, not from an invented student profile.

The earlier ExamAI concept used Java and JSP. This edition uses Python and Flask as its main backend while retaining the examination workflow and AI design. Its separate database, examai_python, avoids modifying the earlier Java project database or the Online Examination Result System database.

3. Objectives

- Build one Python web application that can be presented for both Python and AIP coursework.
- Support teacher and student accounts with role-based access.
- Create, edit and publish timed multiple-choice examinations.
- Use Gemini to draft questions and personalized learning feedback, with explicit validation and teacher review.
- Autosave student answers and enforce the time limit on the server.
- Compute marks reproducibly in Python and store all exam activity in MySQL.

4. Technologies Used

- **Python 3:** application routes, validation, authentication helpers and deterministic scoring.
- **Flask:** web framework for routing, sessions, templates and flash messages.
- **MySQL 8:** relational storage for users, exams, questions, attempts, answers, AI drafts and feedback.
- **PyMySQL:** Python database driver for parameterized SQL statements.
- **HTML, CSS and JavaScript:** responsive pages, question controls, timer display and background answer saving.
- **Gemini REST API:** structured question drafting and grounded post-exam study feedback.

- **Werkzeug:** password hashing and verification used by Flask.

5. Requirements

- Windows 10/11, macOS or Linux; Python 3.10 or newer.
- MySQL Server 8.0 or newer. MySQL Workbench is used to run the setup scripts.
- Internet connection and a valid Gemini API key for live AI requests. Exams and deterministic scoring work without an AI key.
- Python packages listed in requirements.txt.
- Modern browser such as Chrome, Edge or Firefox.

For a local classroom setup, start the MySQL80 service, run 01_schema.sql and 02_create_db_user.sql in Workbench, install the Python requirements, run seed_demo.py, and then start app.py. The supplied SQL does not drop a database.

6. System Design

ExamAI follows a three-tier client–server design. The browser renders Flask templates and sends form or JSON requests. Flask routes enforce the role and ownership rules, coordinate the assessment services and call the data layer. MySQL stores application data. A separate AI service calls Gemini from the server only; the browser never receives the API key.

The data flow diagram shows the student and faculty inputs, the Python application, the MySQL records and the two AI exchanges. The scoring path stays independent from Gemini: the submitted option is checked against the question key, then Python stores the computed score and percentage.

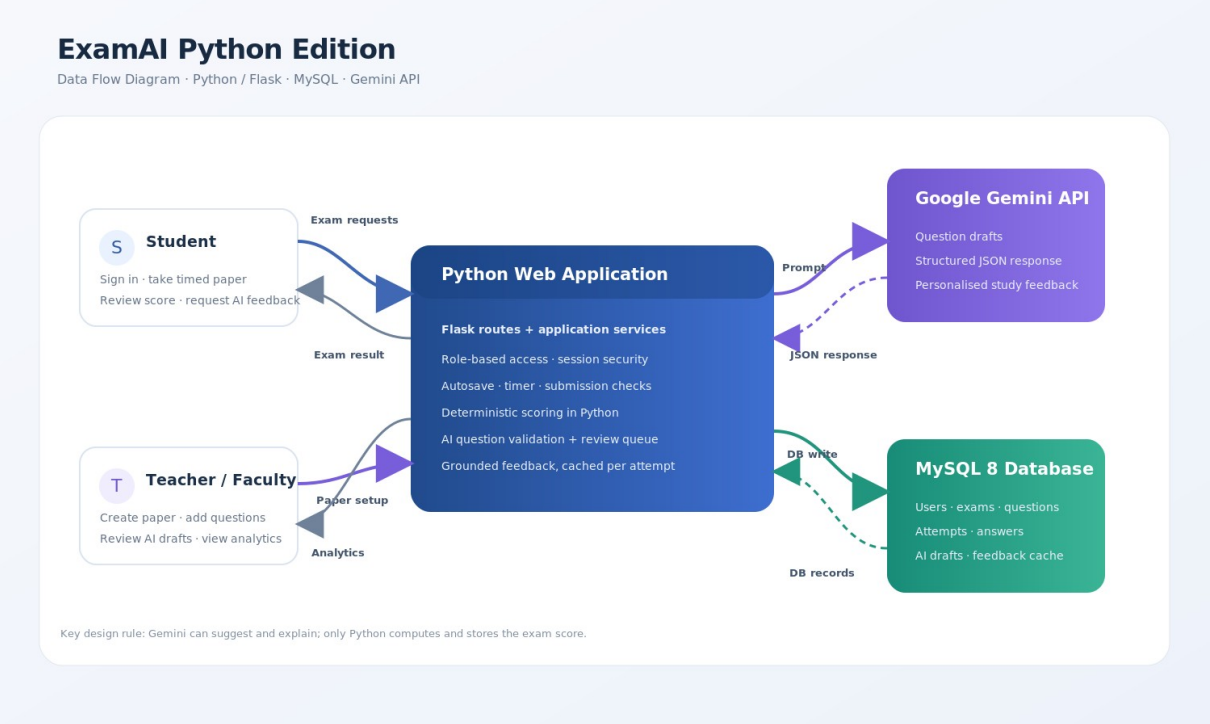


Fig. 1. Data flow of the Python ExamAI application. The model proposes or explains; Python computes marks.

Database design

The schema has seven related tables: users, exams, questions, attempts, answers, ai_drafts and ai_feedback. Foreign keys maintain their relationships. A unique constraint limits a student to one attempt per exam; another keeps one saved response per question in an attempt. Attempts store a frozen question order and server-side deadline.

The following table fragment records the attempt owner, deadline, status and frozen question order. The setup scripts use CREATE IF NOT EXISTS and leave existing project databases untouched.

```
CREATE TABLE IF NOT EXISTS attempts (
    id              INT AUTO_INCREMENT PRIMARY KEY,
    exam_id         INT NOT NULL,
    student_id      INT NOT NULL,
    started_at      DATETIME NOT NULL,
    deadline_at     DATETIME NOT NULL,
    submitted_at    DATETIME NULL,
    score           INT NOT NULL DEFAULT 0,
    total_marks     INT NOT NULL DEFAULT 0,
    percentage      DECIMAL(5,2) NOT NULL DEFAULT 0.00,
    status          ENUM('IN_PROGRESS','SUBMITTED','AUTO_SUBMITTED') NOT NULL DEFAULT 'IN_PROGRESS',
    question_order JSON NOT NULL,
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT uq_attempt_exam_student UNIQUE (exam_id, student_id),
    CONSTRAINT fk_attempt_exam FOREIGN KEY (exam_id) REFERENCES exams(id) ON DELETE CASCADE,
    CONSTRAINT fk_attempt_student FOREIGN KEY (student_id) REFERENCES users(id) ON DELETE CASCADE,
    INDEX idx_attempt_student (student_id),
    INDEX idx_attempt_exam (exam_id),
    INDEX idx_attempt_status (status)
) ENGINE=InnoDB;
```

7. Source Code & Output

Selected listings below are copied from the project source files. The complete Python modules and templates are included with the deliverable. The excerpts show the decision points: deterministic marking, the Gemini request, question validation and the human-review hand-off.

7.1 Deterministic scoring

The score is a pure Python calculation from the question marks, the stored correct option and the saved student choice. A blank answer earns no marks. The function has no Gemini dependency.

```
"""Deterministic marking logic: no model call is involved in awarding marks."""


def calculate_score(question_rows):
    """Return (score, total_marks, percentage) for rows with marks/key/selected_option.

    Each row must contain ``marks`` and ``correct_option``; ``selected_option``
    may be None when the student left a question blank.
    """
    total = sum(int(row["marks"]) for row in question_rows)
    score = sum(
        int(row["marks"])
        for row in question_rows
        if row.get("selected_option") and row["selected_option"] == row["correct_option"]
    )
    percentage = round(score * 100.0 / total, 2) if total else 0.0
    return score, total, percentage
```

7.2 Server-side attempt finalization

```
def finalize_attempt(attempt_id, force=False):
    """Compute the exact score from MySQL answer keys; never trust browser scores or AI."""
    attempt = db.one("SELECT * FROM attempts WHERE id=%s AND student_id=%s",
                     (attempt_id, current_user_id()))
    if not attempt:
        abort(404)
    if attempt["status"] != "IN_PROGRESS":
        return attempt
    now = datetime.now()
    expired = now >= attempt["deadline_at"]
    if not force and not expired:
        return attempt
    # Exact scoring is performed by this Python function from the stored answer key.
    # Gemini is never given authority to award marks.
    rows = db.all_rows(
        "SELECT q.id,q.marks,q.correct_option,a.selected_option FROM questions q "
        "LEFT JOIN answers a ON a.question_id=q.id AND a.attempt_id=%s WHERE q.exam_id=%s",
        (attempt_id, attempt["exam_id"]),
    )
    score, total, pct = calculate_score(rows)
    for row in rows:
        if row.get("selected_option"):
            db.execute("UPDATE answers SET is_correct=%s WHERE attempt_id=%s AND question_id=%s",
                       (row["selected_option"] == row["correct_option"], attempt_id, row["id"]))
```

```
    status = "AUTO_SUBMITTED" if expired else "SUBMITTED"
    db.execute(
        "UPDATE attempts SET score=%s,total_marks=%s,percentage=%s,status=%s,submitted_at=%s WHERE id=%s",
        (score, total, pct, status, now, attempt_id),
    )
    return db.one("SELECT * FROM attempts WHERE id=%s", (attempt_id,))
```

7.3 Gemini request and structured response

The API key is read from the server environment. The response is parsed as JSON and API failures are returned as readable errors. No key is stored in a source file or template.

```
    def _generate_json(self, prompt, schema, temperature=0.4):
        if not self.configured:
            raise AIConfigurationError(
                "Gemini is not configured. Set GEMINI_API_KEY in your local .env file; "
                "no simulated AI response was used."
            )
        models = []
        for model in (self.model, *self.fallback_models):
            if model and model not in models:
                models.append(model)
        body = {
            "systemInstruction": {"parts": [{"text": (
                "You are a careful university assessment assistant. Be accurate, concise, "
                "and return only the structured content requested."
            )}]},
            "contents": [{"role": "user", "parts": [{"text": prompt}]}],
            "generationConfig": {
                "temperature": temperature,
                "responseMimeType": "application/json",
                "responseSchema": schema,
            },
        }
        response = None
        for index, model in enumerate(models):
            url = f"{self.API_ROOT}{model}:generateContent"
            try:
                candidate = self.http.post(
                    url,
                    headers={"x-goog-api-key": self.api_key, "Content-Type": "application/json"},
                    json=body,
                    timeout=(8, 20),
                )
            except requests.Timeout as exc:
                if index + 1 < len(models):
                    continue
                raise AIServiceError("Gemini did not respond before the timeout on the configured or fallback models. Please retry shortly.")
from exc
            except requests.RequestException as exc:
                raise AIServiceError("Could not reach Gemini from this server. Check its internet connection.") from exc

            # If the selected model is under high demand, try an enabled Gemini
            # fallback. No sample response is substituted; only a real API response
            # can be saved as a question or learning review.
            if candidate.status_code == 503 and index + 1 < len(models):
                continue
            if candidate.status_code != 200:
                try:
                    detail = candidate.json().get("error", {}).get("message", "")
                except (ValueError, AttributeError):
                    detail = ""
                if candidate.status_code in (401, 403):
                    message = "Gemini rejected the API key. Check that GEMINI_API_KEY is valid."
                elif candidate.status_code == 404:
                    message = f"Model '{model}' was not found. Set GEMINI_MODEL to an enabled model."
                elif candidate.status_code == 429:
                    message = "Gemini rate limit reached. Wait briefly, then retry."
                elif candidate.status_code >= 500:
                    message = "Gemini is temporarily unavailable after trying the configured fallback models."
                else:
                    message = f"Gemini returned HTTP {candidate.status_code}."
                if detail:
                    message += " " + detail[:220]
                raise AIServiceError(message)
            response = candidate
            self.last_model = model
            break

        if response is None:
            raise AIServiceError("Gemini is temporarily unavailable. Please retry in a moment.")
        try:
            payload = response.json()
            parts = payload["candidates"][0]["content"]["parts"]
            text = "".join(part.get("text", "") for part in parts).strip()
            return json.loads(text)
        except (ValueError, KeyError, IndexError, TypeError) as exc:
            raise AIServiceError("Gemini returned a response that could not be read as the requested JSON.") from exc
```

7.4 Question validation and review queue

```
    @staticmethod
    def validate_question(item):
        if not isinstance(item, dict):
            raise ValueError("question must be an object")
        text = str(item.get("question", "")).strip()
        options = item.get("options")
        idx = item.get("correctIndex")
        if not text or not isinstance(options, list) or len(options) != 4:
            raise ValueError("question and four options are required")
        options = [str(x).strip() for x in options]
        if any(not x for x in options) or len({x.casefold() for x in options}) != 4:
            raise ValueError("options must be non-empty and distinct")
        if isinstance(idx, bool) or not isinstance(idx, int) or idx not in range(4):
            raise ValueError("correctIndex must be between zero and three")
        return {
            "question_text": text[:5000],
            "options": options,
            "correct_option": "ABCD"[idx],
        }
```

The route below saves validated items as pending drafts. A separate teacher action copies an approved question into the exam; an unapproved draft is never presented to a student.

```
        gemini = GeminiService()
        generated = gemini.generate_questions(exam["subject"], topic, difficulty, count)
        for item in generated:
            options = item["options"]
            db.execute(
                "INSERT INTO
ai_drafts(teacher_id,exam_id,topic,difficulty,question_text,option_a,option_b,option_c,option_d,correct_option,explanation,model) "
                "VALUES(%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s)",
                (current_user_id(), exam_id, item["topic"], difficulty, item["question_text"],
                 *options, item["correct_option"], item["explanation"], gemini.last_model),
            )
        if wants_json_response():
            return jsonify(ok=True, count=len(generated), model=gemini.last_model,
                           redirect=url_for("teacher_drafts"))
        flash(f"Gemini generated {len(generated)} draft question(s). Review each before adding it to the exam.", "success")
        return redirect(url_for("teacher_drafts"))
```

7.5 Verification output

The following unit tests ran in the project workspace. They cover scoring, blank answers, JSON parsing, API-key handling, option validation, high-demand fallback and an HTTP rate-limit response. Live teacher question drafting and student learning feedback were also verified using Gemini 3.1 Flash Lite. One validated question draft is pending teacher approval, and one feedback report is cached for a submitted demo attempt. The API key and generated text are not included in this report or the project package.

```
test_http_failure_is_not_silent (test_ai_service.GeminiServiceTests.test_http_failure_is_not_silent) ... ok
test_missing_key_fails_clearly_without_fake_output
(test_ai_service.GeminiServiceTests.test_missing_key_fails_clearly_without_fake_output) ... ok
test_parses_live_api_json_shape_and_sends_key_header
(test_ai_service.GeminiServiceTests.test_parses_live_api_json_shape_and_sends_key_header) ... ok
test_question_validation_maps_correct_index (test_ai_service.GeminiServiceTests.test_question_validation_maps_correct_index)
... ok
test_rejects_duplicate_options (test_ai_service.GeminiServiceTests.test_rejects_duplicate_options) ... ok
test_retries_real_generation_on_a_high_demand_fallback_model
(test_ai_service.GeminiServiceTests.test_retries_real_generation_on_a_high_demand_fallback_model) ... ok
test_tries_fallback_after_primary_model_timeout
(test_ai_service.GeminiServiceTests.test_tries_fallback_after_primary_model_timeout) ... ok
test_all_correct (test_scoring.ScoreTests.test_all_correct) ... ok
test_empty_exam_has_zero_percentage (test_scoring.ScoreTests.test_empty_exam_has_zero_percentage) ... ok
test_score_uses_stored_answer_key_and_ignores_blanks
(test_scoring.ScoreTests.test_score_uses_stored_answer_key_and_ignores_blanks) ... ok

----------------------------------------------------------------------
Ran 10 tests in 0.001s

OK
```

The web-flow smoke test connected to MySQL 8.0.46 and exercised teacher and student routes, question creation and editing, publication, answer autosave, submission and result display. The health endpoint reported that the database connection was available.

```
[2026-10-26 10:31:35,644] WARNING in app: Gemini student-feedback request failed: AIConfigurationError
PASS teacher login and role-protected dashboard
PASS teacher creates a draft exam
PASS manual question validation and insertion
PASS editing a question in an unused draft
```

PASS deleting and re-adding a draft question
PASS publication requires a question and makes the paper available
PASS student login and role-protected dashboard
PASS student starts a timed attempt
PASS answer autosave with CSRF protection
PASS submission and deterministic Python score: 2/2 (100.00%)
PASS result review; unavailable Gemini key is reported clearly, no fake feedback
PASS health check: MySQL connected; live AI configured=False
CLEANUP temporary smoke-test exam and attempt

Screens from the running application

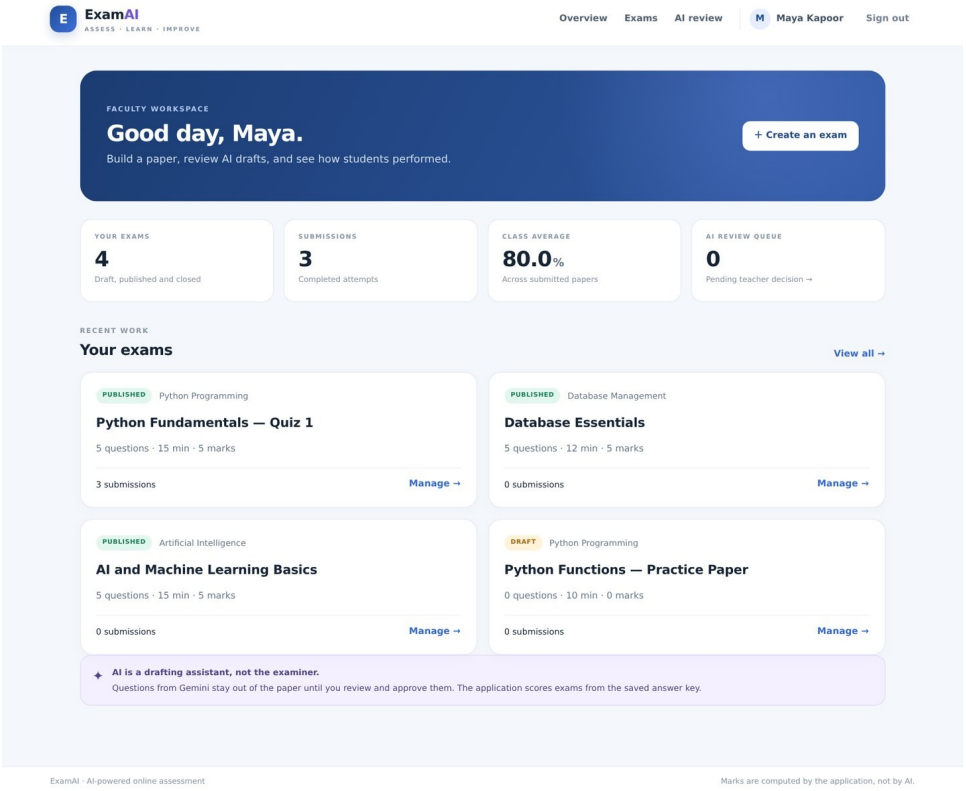


Fig. 2. Faculty dashboard with exam and submission statistics from the local MySQL data.

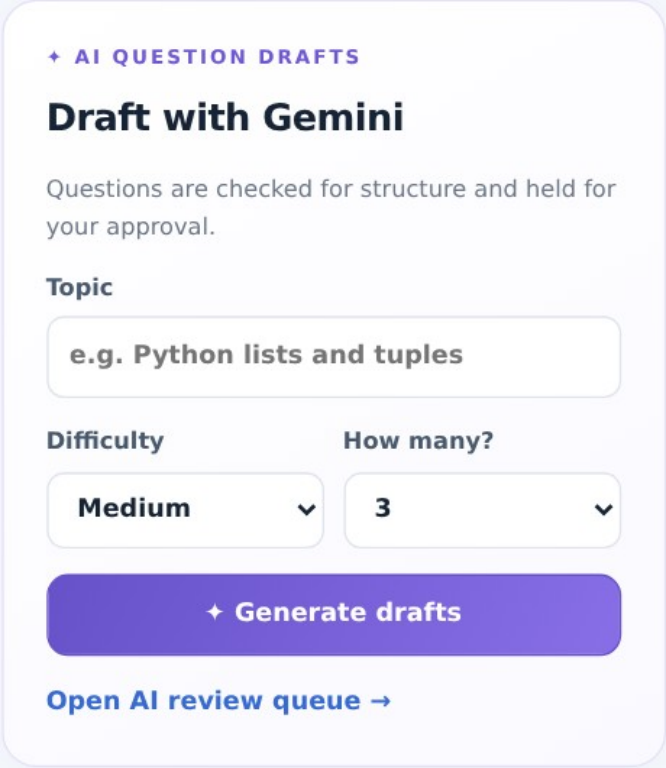


Fig. 3. Gemini question-drafting panel. Generated items enter a review queue and are not published automatically.

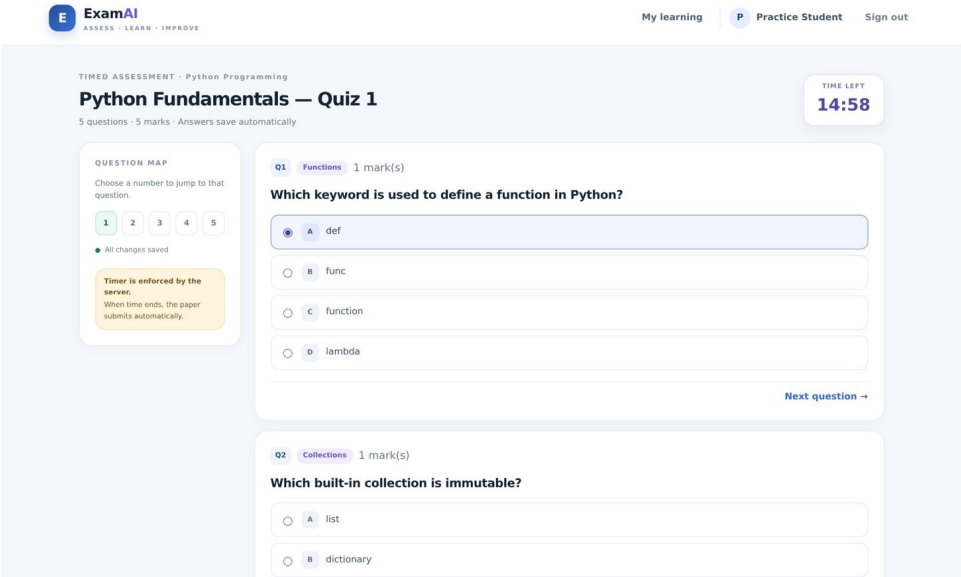


Fig. 4. Timed assessment. The answer is autosaved and the countdown is displayed.

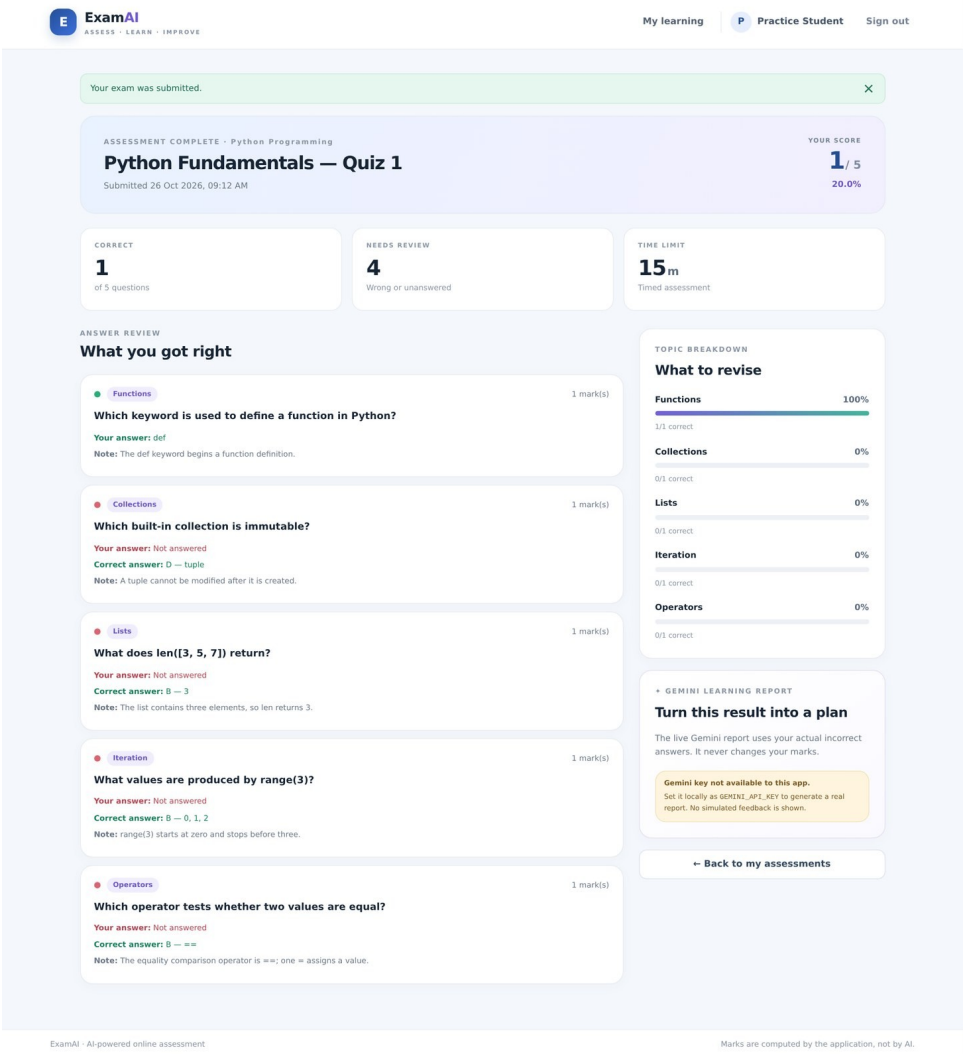


Fig. 5. Result review from the default no-key capture state. The score is computed from the answer key; live Gemini feedback needs a key and network access.

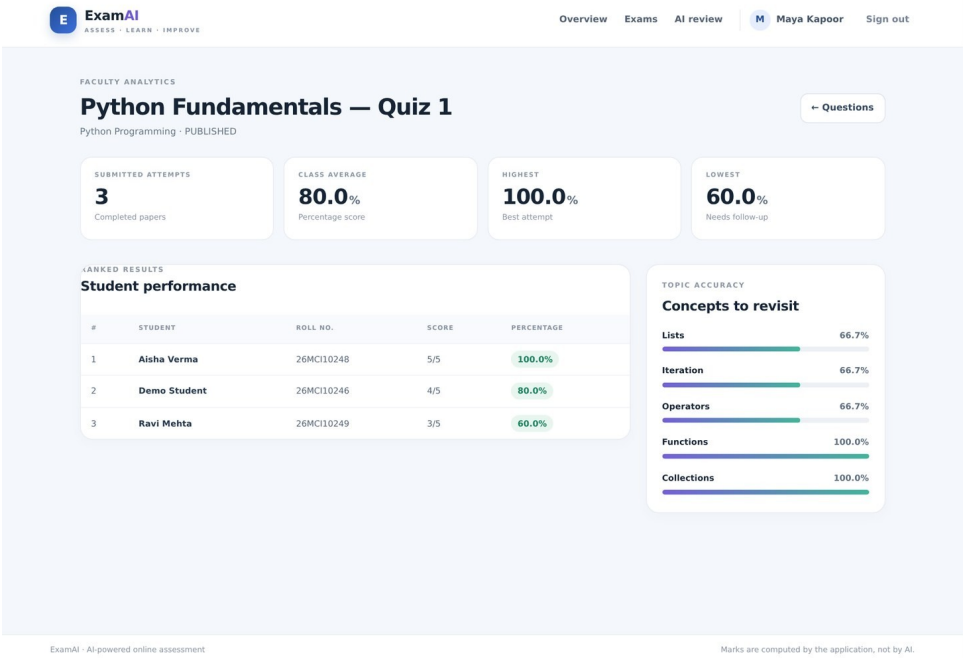


Fig. 6. Faculty analytics showing submitted scores and topic accuracy.

8. Functionalities

Faculty

- Create exams and manage their draft, published and closed states.
- Add, edit and delete questions before the exam has attempts.
- Request Gemini question drafts by topic and difficulty; review, approve or reject each draft.
- View attempt counts, class average, highest and lowest scores, ranked student performance and topic accuracy.

Student

- Register as a student and view published assessments.
- Start a single timed attempt; the server checks the deadline even if the browser timer is manipulated.
- Autosave selected options and resume an in-progress attempt.
- Review the score, selected and correct answers, explanations and accuracy by topic.
- Request AI study feedback grounded in the actual incorrect answers when Gemini is configured.

AI safeguards

- AI output is validated for a non-empty question, four distinct options and a valid correct-answer index.
- Generated items remain in a pending queue until approved by the teacher.
- The model is never asked to calculate or award marks.
- AI feedback is cached per attempt and uses question-level evidence rather than names or emails.

9. Conclusion

The Python ExamAI edition combines an operational online examination workflow with a genuine server-side Gemini integration. Flask handles the roles and requests; MySQL preserves the paper and attempt data; Python computes the score; Gemini can draft questions and explain mistakes. The human review queue and deterministic marking separate AI assistance from academic decisions.

The project was exercised against MySQL 8.0.46 with demo accounts and sample assessments. Unit and web-flow checks passed, and live Gemini question drafting and student feedback were verified in the hosted environment. The distributable project contains no API key; a local installation needs a private key in its own .env file for Gemini requests.

10. Future Work

- Add a teacher-controlled pass threshold and exportable PDF mark sheets.
- Support more question types, including numerical and short-answer responses with rubric-based teacher grading.
- Add exam availability windows and reminders.
- Add attempt audit logs and richer course-level analytics.
- Deploy behind HTTPS with production secret management, backups and privacy controls.

11. Learning Outcomes

- Building Python web routes, templates, sessions and role checks with Flask.

- Using parameterized PyMySQL queries and relational constraints in MySQL 8.
- Separating server-side score calculation from the browser and from AI calls.
- Validating structured generative-AI output before it is saved or shown to a student.
- Implementing a teacher-in-the-loop review workflow for AI-generated educational content.
- Testing with unit tests and an end-to-end flow rather than relying only on a screenshot.

12. GitHub Link

[ADD YOUR REPOSITORY LINK HERE]

13. Bibliography

[1] Pallets Projects. Flask Documentation. https://flask.palletsprojects.com/

[2] Python Software Foundation. Python 3 Documentation. https://docs.python.org/3/

[3] MySQL. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

[4] PyMySQL Project. PyMySQL Documentation. https://pymysql.readthedocs.io/

[5] Google AI for Developers. Gemini API Documentation. https://ai.google.dev/gemini-api/docs

[6] Google AI for Developers. Gemini API Models. https://ai.google.dev/gemini-api/docs/models

[7] OWASP Foundation. Cross Site Request Forgery Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html